

Rapport Projet Génie Logiciel

Groupe GI3 A: CRISSOT Martin - KOROGHLI Adlène - DUVOYÉ Alois
JAIDI Ismail - HIDA Yasmina

Thématique 3 : Déplacement d'agents dans un graphe avec une limitation du nombre
d'agents par arête

ING1 · 2025-2026

Tuteur : M. HADDACHE Mohammed

Projet Génie Logiciel



SOMMAIRE

1. Introduction
2. Présentation de l'équipe
3. Organisation du travail et flux de développement
4. Problèmes rencontrés et solutions apportées
5. Fonctionnalités implémentées
6. Diagramme UML (diagramme de classes)
7. Cas d'utilisation
8. Résultats et démonstration
9. Conclusion et perspectives
10. Annexes

1. Introduction

Ce projet a pour objectif de développer une application graphique permettant de visualiser, modifier et simuler le déplacement d'agents dans un graphe. Chaque agent possède des propriétés individuelles (vitesse, mode de déplacement, destination, tolérance à la congestion, etc.). L'application offre une interaction visuelle en temps réel sur une carte 2D, avec la possibilité d'observer les modifications en direct.

Notre projet se focalise spécifiquement sur l'utilisation du graphe pour modéliser et gérer des systèmes d'évacuation d'incendie dans une gare. Les nœuds représentent des points stratégiques (quais, couloirs, sorties, escaliers, halls), et les arêtes symbolisent les liaisons entre ces zones (passages, portes, couloirs). Les agents incarnent des voyageurs ou des personnels, chacun avec des comportements propres (calme, paniqué, priorité, etc.).

L'application permet de :

- Simuler la propagation d'un incendie (déclenchement de feux sur certains nœuds/arêtes)
- Observer les réactions des agents : changement de comportement, évitement des zones dangereuses, recherche de sorties de secours
- Analyser l'efficacité des plans d'évacuation (temps d'évacuation, goulots d'étranglement, zones à risque)
- Tester différents scénarios (nombre d'agents, largeur des couloirs, orientation des flux, etc.)

Le projet s'inscrit dans une démarche intégrant les principes d'Éthique et de Design travaillés en amont par l'équipe. Il répond à un besoin concret de sécurité des personnes dans les infrastructures ferroviaires, en fournissant un outil d'aide à la décision pour les gestionnaires de gares et les services d'urgence.

2. Présentation de l'équipe

L'équipe est composée de cinq membres :

Nom	Rôle principal
Yasmina HIDA	Création des types d'agents + interface console
Aloïs DUVOYE	Gestion de la simulation + création des éléments du graph + pathfinding + diagramme de classe
Adlène KOROGHLI	Création du graphe + gestion de la simulation
Martin CRISSOT	Gestion des agents et de leurs comportements + gestion de la congestion dans les arêtes + diagramme de classe et de cas d'usages
Ismail JAIDI	Rédaction du rapport + interface graphique

L'équipe a utilisé Microsoft Teams comme outil de communication principal, et GitHub pour le suivi des tâches et la gestion du code. Le développement a été réalisé sous VS Code et Eclipse en utilisant JavaFX pour la partie visuelle.

Un fonctionnement basé sur des objectifs hebdomadaires clairs et concrets a été mis en place, avec une entraide active entre membres en cas de besoin. Des réunions avec le tuteur de projet ont eu lieu deux fois par semaine pour suivre l'avancement.

3. Organisation du travail et flux de développement

3.1 Méthodologie

L'équipe a adopté une approche itérative avec des cycles de développement courts. Chaque semaine, des objectifs précis étaient définis et répartis. Les membres pouvaient s'entraider sur les tâches bloquantes en cas de besoin ou d'indisponibilité de certains membres.

3.2 Gestion de version (Git)

- Dépôt : https://github.com/Yotacaz/PGL_GI3A (privé)
- Fréquence des commits : au moins 1 commit par jour en moyenne, conformément au cahier des charges.

- Stratégie de branches : plusieurs branches de travail ont été utilisées pour découper les tâches .
- Revues : validation croisée avant fusion sur la branche principale.
- Suivi : les tâches ont été suivies via GitHub (issues, projets).

3.3 Planification et suivi

Des réunions internes régulières ont permis de :

- Répartir les tâches
- Résoudre les blocages techniques
- Vérifier l'avancement par rapport aux jalons
- Garder une cohésion d'équipe nécessaire au bon fonctionnement du projet

Contrainte rencontrée : Certains membres ont été temporairement indisponibles, nécessitant une redistribution des tâches et une prise en charge de plusieurs rôles par certains contributeurs. De plus, le délai imparti s'est avéré insuffisant pour un projet de cette envergure. Un démarrage plus tôt aurait permis une meilleure planification et une qualité supérieure .

4. Problèmes rencontrés et solutions apportées

4.1 Problèmes techniques

Problème	Solution
Prise en main de JavaFX au début du projet	Temps d'apprentissage collectif, tests sur des petites interfaces avant intégration
Gestion de la congestion et des arêtes orientées	Implémentation d'un système de capacité par arête et de files d'attente
Création d'une interface graphique ergonomique et lisible	Itérations sur le design, retours croisés entre membres, définitions d'une charte graphique
Comportements particuliers des agents (évitement de zones denses, priorité, etc.)	Modularisation des stratégies de déplacement dans des classes dédiées

Gestion des exceptions pour éviter les crashes	Mise en place d'un système de gestion d'erreurs générique avec affichage utilisateur adapté
Choix de l'algorithme de pathfinding	• Dijkstra (distance pure) — utilisé par <code>nearestExit()</code> pour identifier rapidement la sortie géographiquement la plus proche, sans tenir compte de l'état dynamique du graphe. • Multi-Agent Pathfinding (appelé MAPF dans le code), utilisé par <code>shortestPath()</code> pour calculer le chemin optimal à chaque décision d'agent. Le coût d'une arête combine : la distance euclidienne, une pénalité de congestion sur l'arête et le nœud destination (facteur 0.5), une forte pénalité feu (1000), et une pénalité de gabarit si l'arête est trop étroite pour l'agent (5000). Cela permet aux agents d'éviter dynamiquement les zones en feu et les couloirs saturés.
refactoring complexe qui a introduit des bugs	Utilisation de beaucoup plus de méthodes de test sur les paramètres d'entrée et parfois sur les valeurs calculées.
Comportement des agents incohérent et pas uniformes	Évaluation de plus de paramètres pour les décisions, utilisation d'exponentielles, normalisations de certaines valeurs entre 0 et 1.

4.2 Problèmes d'organisation

Comme mentionné plus haut, l'indisponibilité ponctuelle de certains membres a été gérée par une réaffectation temporaire des tâches et un renforcement de l'entraide. Aucun retard majeur n'a été constaté.

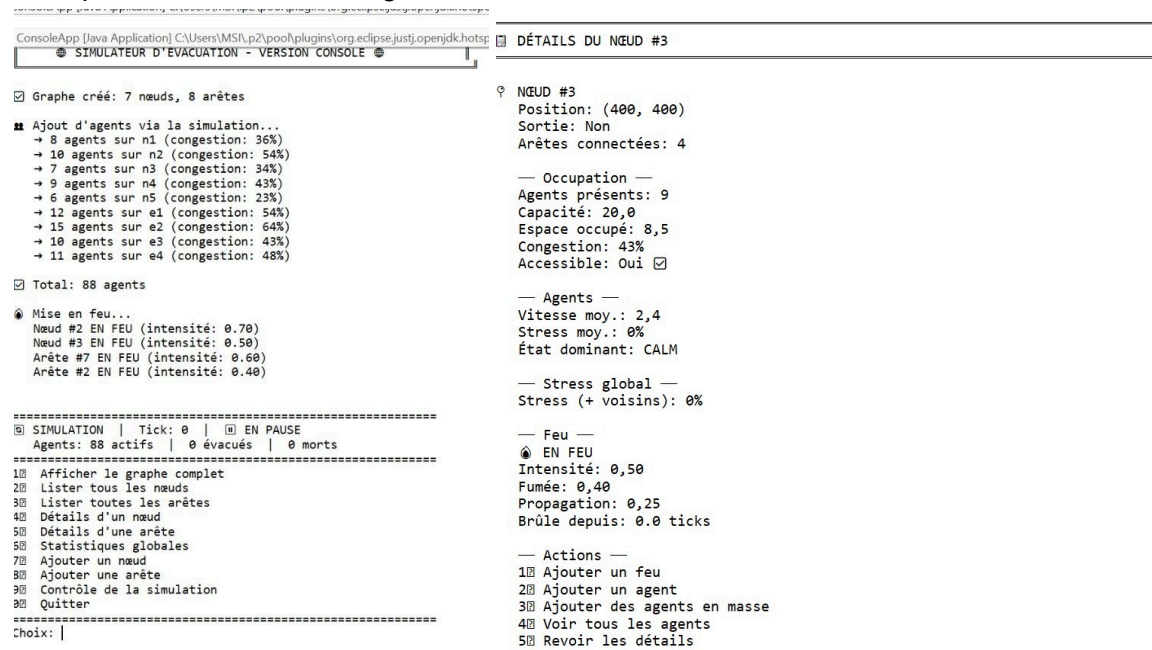
5. Fonctionnalités implémentées

5.1 Version en ligne de commande

La version CLI est totalement fonctionnelle. Elle permet de :

- Créer et gérer le graphe : Visualiser la structure complète , lister tous les nœuds et arêtes, ajouter de nouveaux nœuds et arêtes
- Ajouter et distribuer des agents : Ajouter des agents avec répartition automatique et suivi des taux de congestion par zone
- Configurer les conditions d'urgence : mettre en feu des nœuds et arêtes avec contrôle d'intensité
- Lancer et contrôler la simulation : Mettre en pause, visualiser le statut en temps réel (agents actifs, évacués, morts)
- Consulter les détails et statistiques: Afficher les informations complètes de chaque nœud/arête et les statistiques globales
- Navigation interactive : Menu numéroté permettant d'accéder facilement à chaque fonction

Quelques screenshots témoignant du fonctionnement de la version console :



5.2 Version graphique (JavaFX)

Toutes les fonctionnalités demandées sont opérationnelles :

- Interface ergonomique et lisible

- Affichage du graphe (nœuds et arêtes orientées)
- Modification en direct : ajout, suppression, déplacement de nœuds et arêtes pendant la simulation
- Ajout de masse : création de X nœuds aléatoires
- Gestion des agents :
 - Ajout / suppression manuelle ou en masse
 - Modification des propriétés des agents
 - Déclenchement de feux dans les nœuds et arêtes → les agents peuvent réagir à ces événements (changement de comportement, évacuation)
- Simulation temporelle : vitesse réglable, pause, pas à pas
- Gestion des congestions : capacité des arêtes, files d'attente, ralentissements
- Pathfinding : algorithme de Dijkstra modifié pour trouver les sorties les plus proches associé à un pathfinding MAPF (Multi-agent Pathfinding) qui utilise une pondération distance + pénalité feu + pénalité congestion .
- Statistiques :
 - Vitesse moyenne des agents dans un nœud
 - Vitesse moyenne des agents dans une arête
- Propriétés modifiables :
 - Largeur d'une arête (capacité)
 - Longueur d'une arête (distance)
- Sauvegarde / Restauration : sérialisation binaire complète (graphe + agents + état des feux)

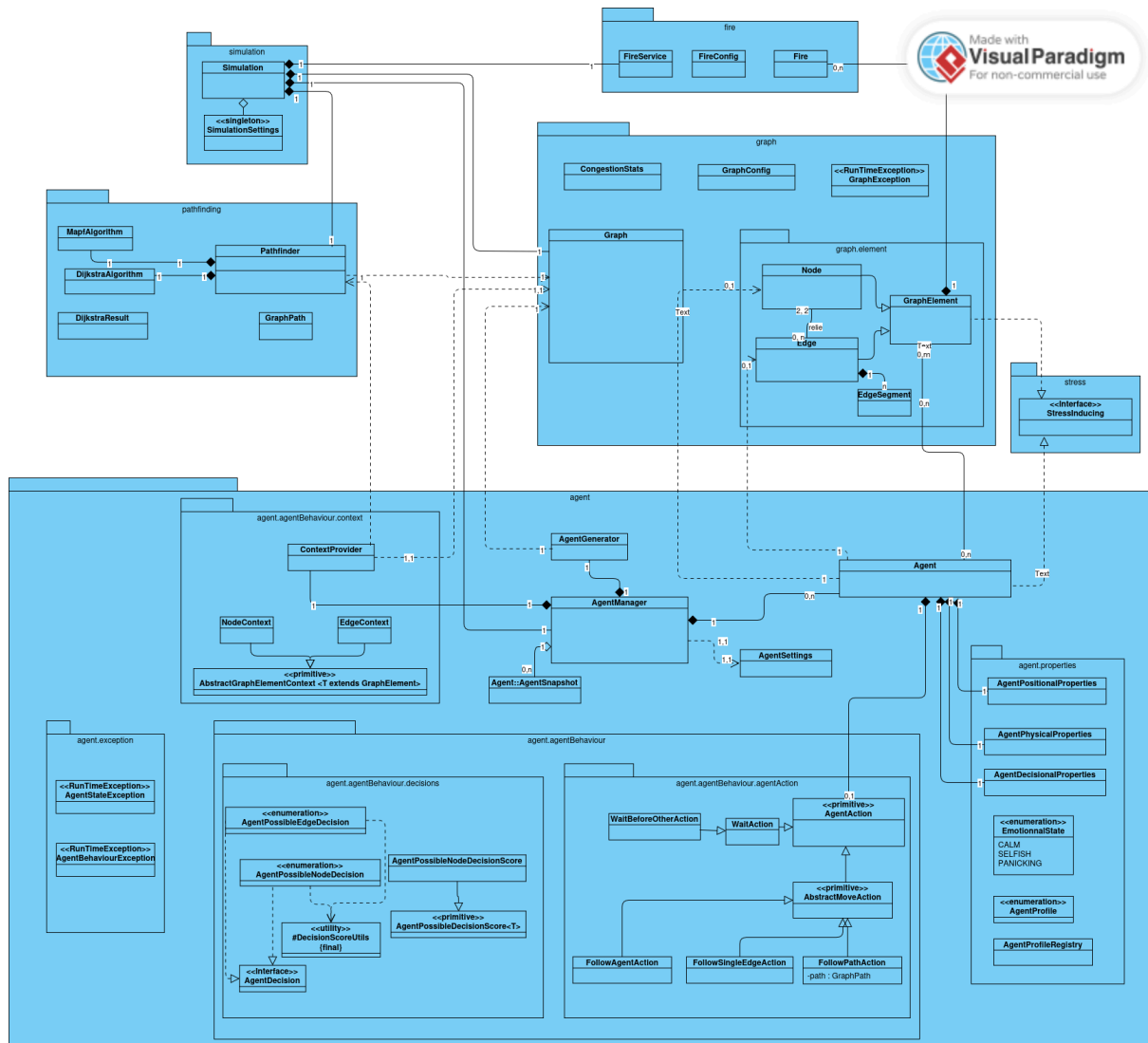
5.3 Comportements particuliers des agents

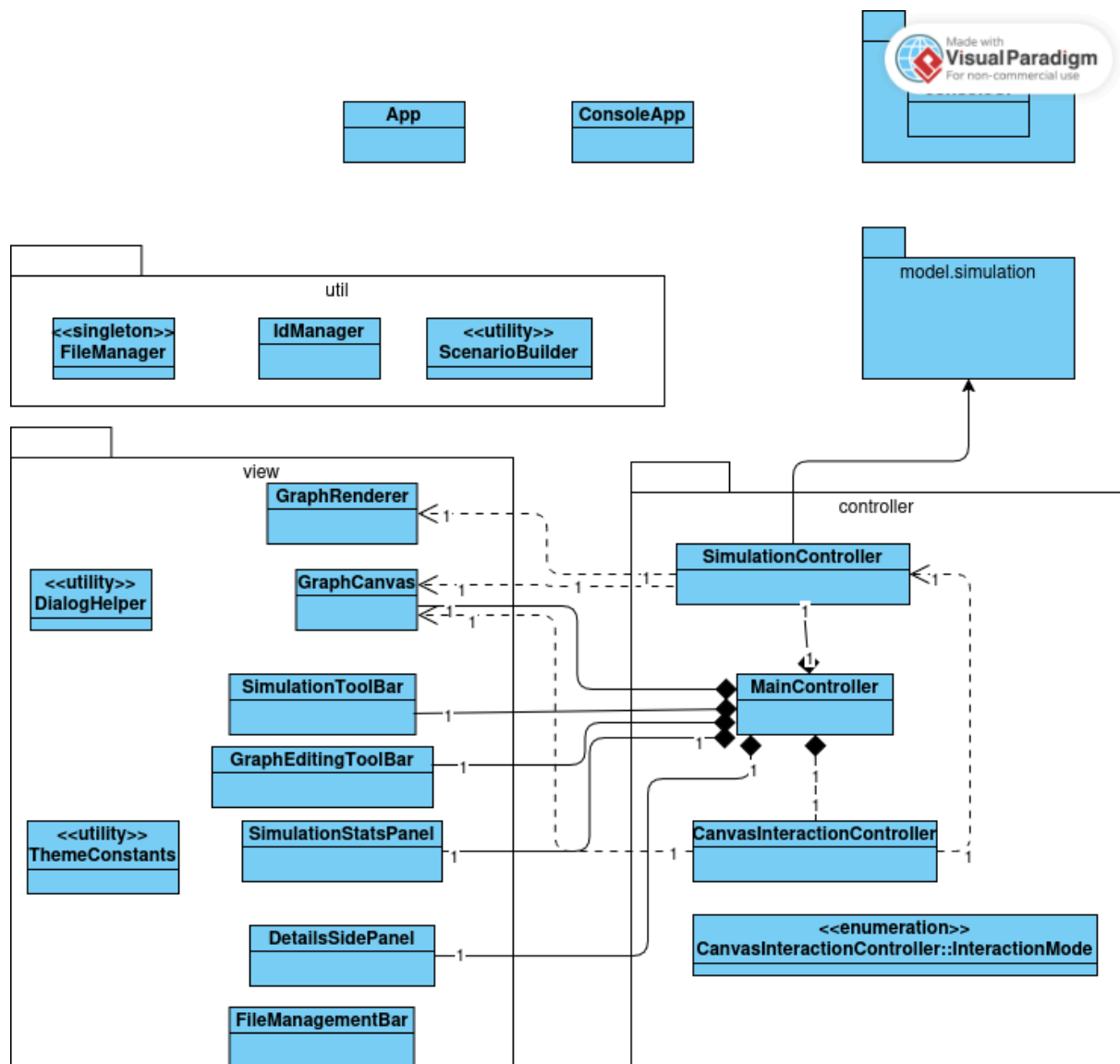
- CALM → SELFISH → PANICKING : les agents changent d'état émotionnel en fonction de leur niveau de stress (lui-même influencé par la congestion locale, le feu, le niveau de stress des agents autour de lui). Un agent paniqué prend des décisions moins rationnelles, se déplace en courant, et prête moins attention aux consignes (direction des arêtes).
- Les agents prennent des décisions quand ils sont sur un nœud ou quand ils sont sur une arête qu'ils transforment en une action correspondante. Les agents avec le facteur de décision le plus élevé décident en premier et influencent les autres.
- Évitement du feu/congestion : le **PathFinder** évalue un coût de trajet tenant compte du niveau de feu et de la congestion des arêtes. Dans le cas idéal, l'agent choisit le chemin de coût minimal qui est conseillé par notre algorithme — s'il voit une arête en feu, il tente de contourner.
- Grâce à une congestion évaluée de manière locale à chaque arête, si beaucoup d'agents empruntent le même couloir au même endroit sur l'arête, leur vitesse sera grandement ralentie. Quand des agents vont dans les deux sens d'une arête, le ralentissement provoqué sera encore plus important

5.4 Système d'évacuation intelligent

L'application intègre une fonctionnalité avancée de repérage des zones à risque dans un bâtiment (détection des nœuds/arêtes en feu ou dangereux). Les agents peuvent adapter leur trajet pour évacuer vers des sorties sûres, en évitant les zones dangereuses dynamiquement. Pour cela on utilise notamment l'algorithme MAPF (Multi Agent Pathfinding) qui permet de gérer les congestions de la manière la plus efficace.

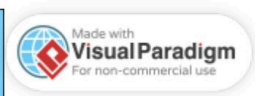
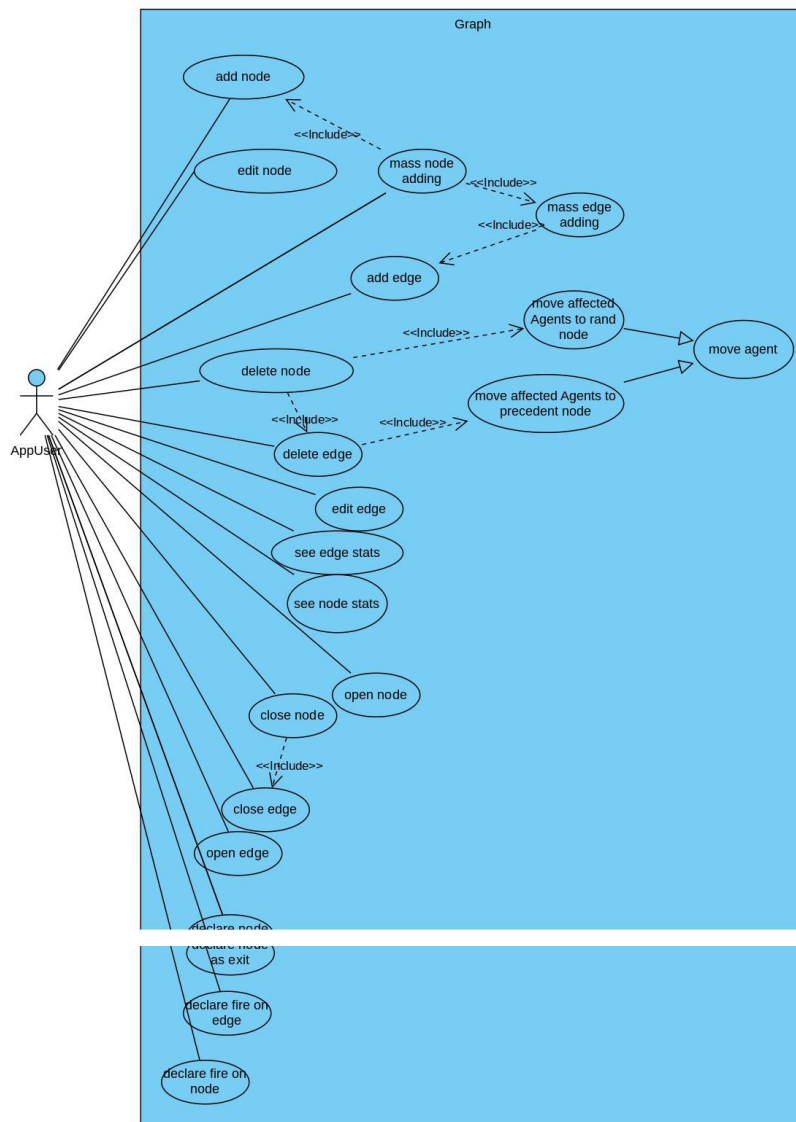
6. Diagramme UML (diagramme de classes)

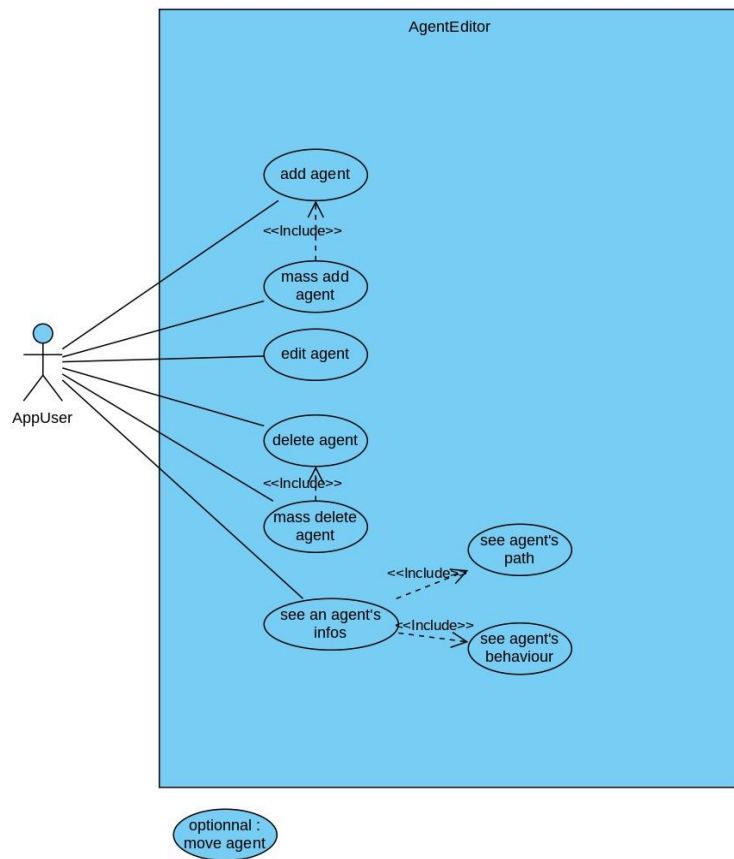


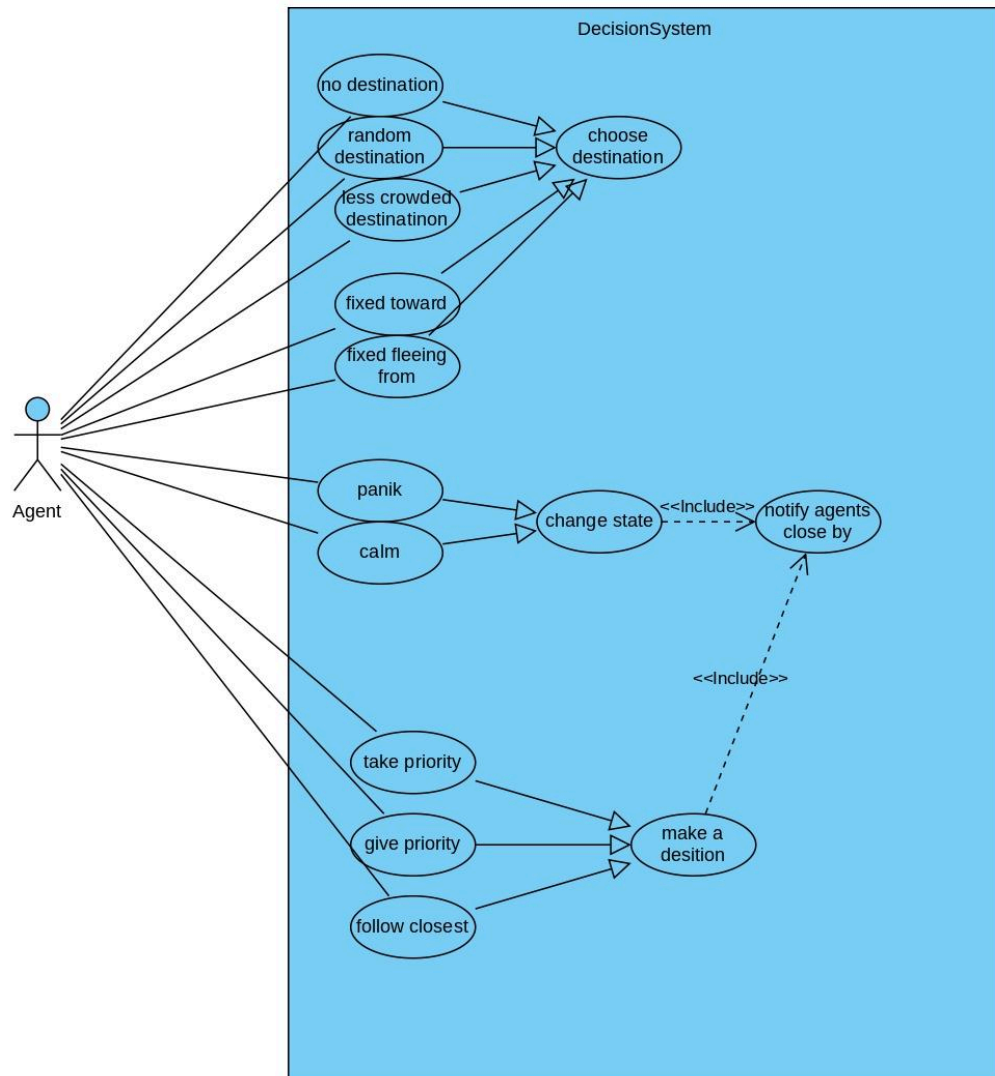


nb: les attributs et les méthodes n'ont pas été mis car ils rendaient le diagramme beaucoup trop lourd

7. Cas d'utilisation







8. Résultats et démonstration

8.1 État final de l'application

Ce qui fonctionne complètement :

- Toutes les fonctionnalités listées en section 5 (version CLI et graphique)
- Création/suppression de nœuds, arêtes, agents (manuel et en masse)
- Déclenchement de feux et réaction des agents
- Statistiques (vitesse moyenne)
- Modification de la largeur/longueur des arêtes
- Sérialisation complète (sauvegarde/restauration)

Ce qui est partiel ou manquant :

- Rien de significatif , la version finale est entièrement fonctionnelle.

8.2 Stabilité et gestion des erreurs

L'application ne crash jamais intempestivement. Toutes les exceptions sont capturées et gérées proprement (affichage d'un message à l'utilisateur sans interruption du programme). Les boucles infinies sont impossibles grâce à une gestion rigoureuse des conditions de sortie.

8.3 Démonstration prévue

La démonstration se fera :

- En ligne de commande (CLI) pour montrer la robustesse du modèle métier
- Via l'interface graphique JavaFX pour l'interaction utilisateur
- Ou bien éventuellement via le terminal avec une version console

Deux machines seront utilisées comme demandé dans le cahier des charges. L'équipe est prête à effectuer des modifications en direct du code si le jury le demande.

9. Conclusion et perspectives

9.1 Bilan

L'équipe a réussi à livrer une application complète, stable, conforme au cahier des charges et enrichie de fonctionnalités avancées (gestion des incendies, évacuation intelligente). La principale difficulté technique surmontée a été la spécialisation du comportement des différents agents.

La force de l'équipe a été sa capacité à répartir le travail efficacement et à s'adapter aux indisponibilités de certains membres sans perte de productivité. L'utilisation de GitHub (branches, issues) a grandement facilité le travail collaboratif.

9.2 Améliorations possibles

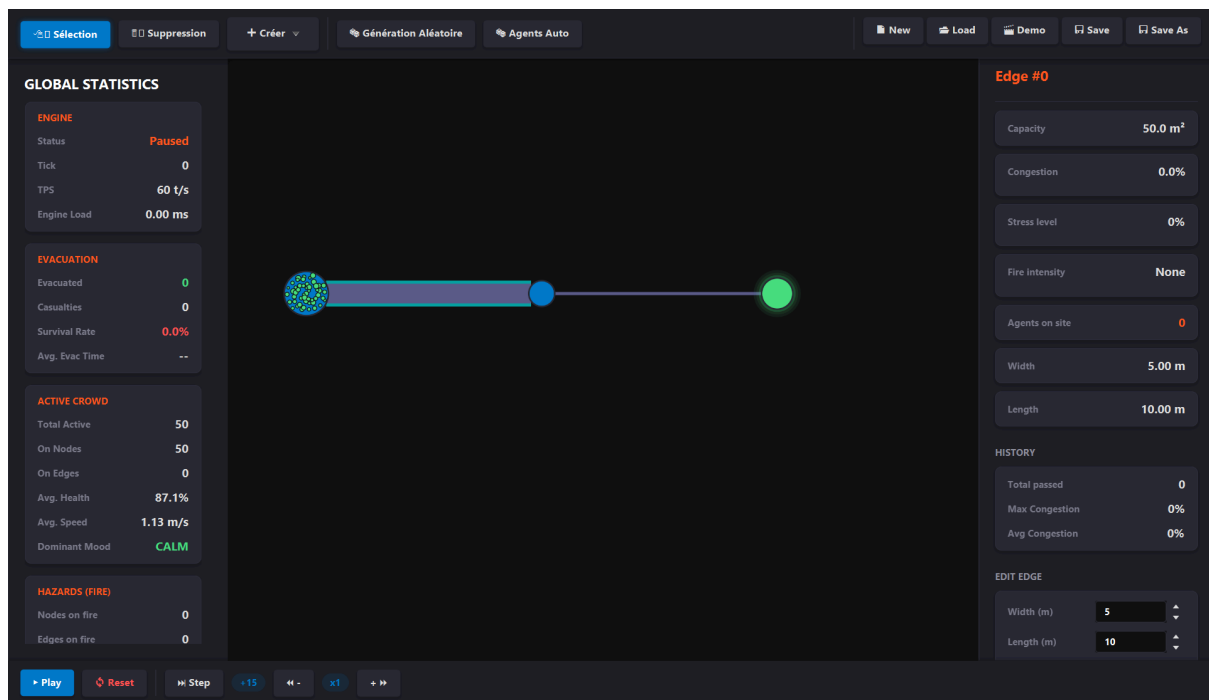
Si le projet devait être prolongé, les améliorations suivantes pourraient être apportées :

- Implémentation de plus de comportements d'agents (panique, contagion émotionnelle, priorité absolue)
- Export de statistiques vers CSV/JSON pour analyse externe

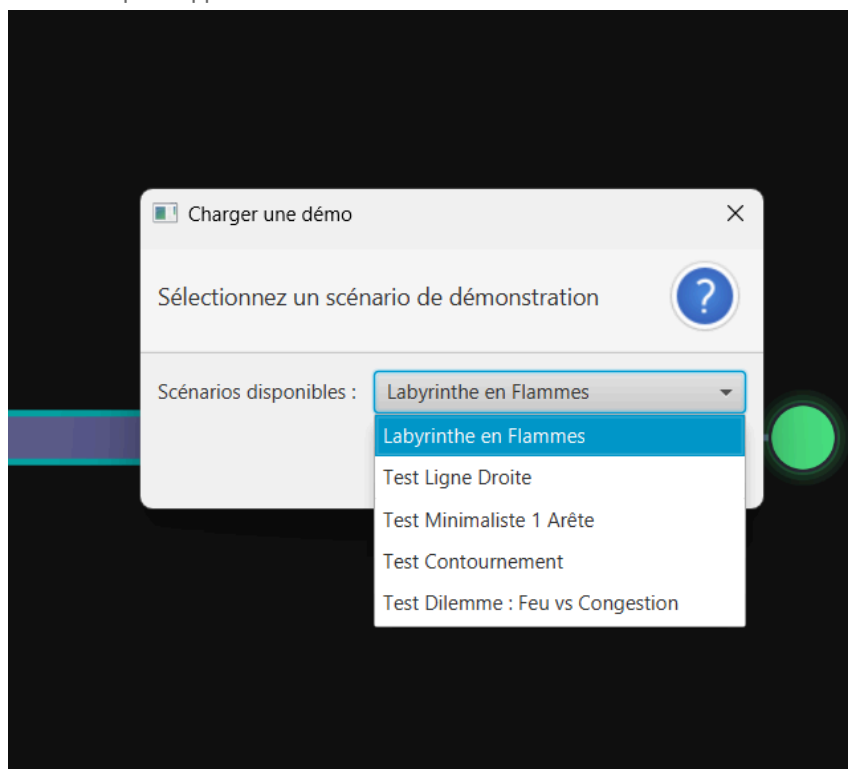
10. Annexes

- Lien dépôt Git : https://github.com/Yotacaz/PGL_GI3A (privé)

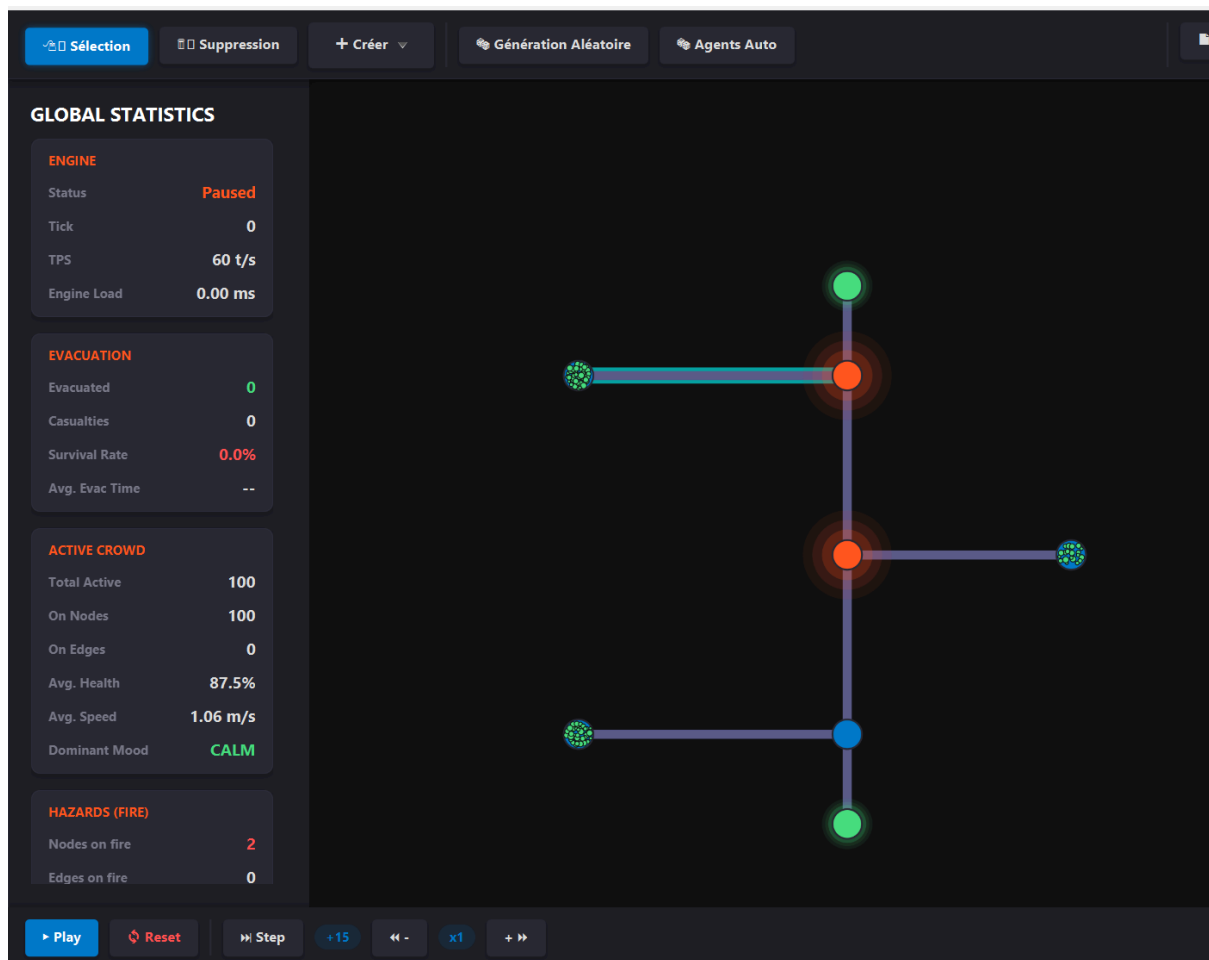
10.1 Captures d'écran



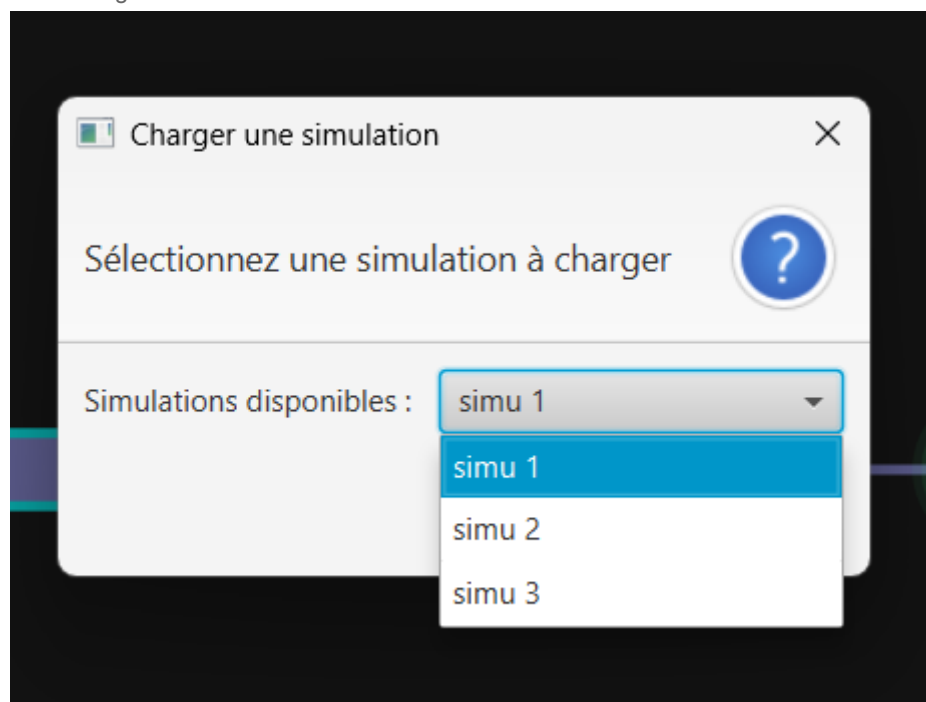
exemple standard de ce à quoi l'application ressemble



Les différentes démo chargeable



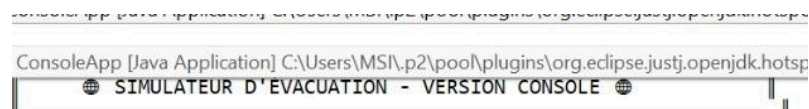
exemple de demo chargeable



exemple de l'utilisation de la sauvegarde / import de simulations



Logo de l'application



☒ Graphe créé: 7 nœuds, 8 arêtes

■ Ajout d'agents via la simulation...

- 8 agents sur n1 (congestion: 36%)
- 10 agents sur n2 (congestion: 54%)
- 7 agents sur n3 (congestion: 34%)
- 9 agents sur n4 (congestion: 43%)
- 6 agents sur n5 (congestion: 23%)
- 12 agents sur e1 (congestion: 54%)
- 15 agents sur e2 (congestion: 64%)
- 10 agents sur e3 (congestion: 43%)
- 11 agents sur e4 (congestion: 48%)

☒ Total: 88 agents

● Mise en feu...

- Nœud #2 EN FEU (intensité: 0.70)
- Nœud #3 EN FEU (intensité: 0.50)
- Arête #7 EN FEU (intensité: 0.60)
- Arête #2 EN FEU (intensité: 0.40)

```
=====
[SIMULATION] | Tick: 0 | [EN PAUSE]
Agents: 88 actifs | 0 évacués | 0 morts
=====
1 [Afficher le graphe complet]
2 [Lister tous les nœuds]
3 [Lister toutes les arêtes]
4 [Détails d'un nœud]
5 [Détails d'une arête]
6 [Statistiques globales]
7 [Ajouter un nœud]
8 [Ajouter une arête]
9 [Contrôle de la simulation]
0 [Quitter]
=====
Choix: |
```

📄 DÉTAILS DU NŒUD #3

📍 NŒUD #3

Position: (400, 400)
Sortie: Non
Arêtes connectées: 4

— Occupation —

Agents présents: 9
Capacité: 20,0
Espace occupé: 8,5
Congestion: 43%
Accessible: Oui ☒

— Agents —

Vitesse moy.: 2,4
Stress moy.: 0%
État dominant: CALM

— Stress global —

Stress (+ voisins): 0%

— Feu —

🔥 EN FEU
Intensité: 0,50
Fumée: 0,40
Propagation: 0,25
Brûle depuis: 0.0 ticks

— Actions —

- 1🔍 Ajouter un feu
- 2👤 Ajouter un agent
- 3👥 Ajouter des agents en masse
- 4👁 Voir tous les agents
- 5🔍 Revoir les détails
- 6🔍 Détails